

The Verified-Candidate Loop for Budgeted Large Language Model-Assisted High-Level Synthesis

Anonymous Submission

Abstract—High-level synthesis (HLS) agents must repair and optimize unfamiliar kernels within limited large language model (LLM) and tool budgets. We introduce the Verified-Candidate Loop (VCL), which grants candidate-promotion authority to an ordered validation chain. The agent ranks gate-valid candidates with a quality-of-results (QoR) score that combines effective latency and worst-resource growth. QoR-guided retrieval-augmented generation (QoR-RAG) retrieves compatible rules, successes, and measured failures. Failure Reflection converts key diagnostics and source changes into the next constraint without another LLM request. The agent records every gate, tool call, and token for reproducible evaluation.

Index Terms—high-level synthesis, LLM agent, FPGA, retrieval-augmented generation, quality of results

I. EVIDENCE MUST CONTROL LLM EDITS

HLS engineers must interpret unfamiliar C/C++ kernels, fix functional and streaming failures, and tune hardware under a small evaluation budget. Vitis HLS exposes correctness and performance only through simulation and synthesis reports [1]. An LLM can propose source changes, but its text cannot certify an interface, a register-transfer-level (RTL) execution, or a hardware tradeoff.

LLM-assisted HLS systems automate code transformation and design-space exploration [2], [3]. General tool-using agents interleave model reasoning and actions [4]. A metered HLS loop additionally requires a candidate-promotion rule. An unchecked edit can pass compilation while failing a hidden test, deadlocking in RTL, missing 100 MHz, or growing one resource beyond its speedup. Partial validation can also exhaust the credits needed to recover.

We call our evidence-controlled promotion mechanism the Verified-Candidate Loop (VCL). The paper develops three claims. **(1) Validity preservation:** the validation chain prevents candidates that fail any available required gate from replacing the verified fallback. **(2) QoR alignment:** the bounded hardware-quality score Q_{HW} rejects speedups that lose on clock or the worst-growing resource. **(3) Evidence reuse:** QoR-guided retrieval-augmented generation (QoR-RAG) and Failure Reflection focus later iterations on compatible actions and prior failures. Section V maps each claim to a separate experiment; the final revision will add measured results.

II. THE VCL PRESERVES THE VERIFIED FALLBACK

The VCL separates proposal, validation, and promotion. Task preflight records the editable kernel, public tests, exact top-function signature, target part, and budgets. The LLM proposes one complete source candidate. The validation chain starts with an interface gate that checks required includes, the

signature, balanced source structure, and forbidden dependencies.

The remaining gates run C simulation (CSIM), HLS synthesis (SYNTH), the 100 MHz frequency check, the U55C capacity check, and required C/RTL co-simulation (COSIM). The metric-completeness gate requires latency, clock, and all five resource counts. Let each applicable gate $G_i \in \{0, 1\}$; an inapplicable COSIM gate equals one. The candidate validity is

$$V(c) = G_{\text{int}} G_{\text{csim}} G_{\text{syn}} G_{\text{freq}} G_{\text{cap}} G_{\text{cosim}} G_{\text{metric}}. \quad (1)$$

Let b denote the verified fallback. The promotion rule is

$$P(c) = V(c) \wedge [Q_{\text{HW}}(c) > Q_{\text{HW}}(b)]. \quad (2)$$

Functional and synthesis failures enter the repair path. A COSIM failure enters the structural path, whose prompt targets interleaved producer writes, bounded stream depths, and producer-consumer rate balance. Before candidate tools run, budget admission requires enough credits for CSIM, SYNTH, and required COSIM. A failed gate or budget denial retains b , the latest kernel that passed every available required gate. Hidden acceptance remains an external evaluator check.

Figure 1 maps the promotion boundary and feedback path.

III. QoR REJECTS FALSE SPEEDUPS

Q_{HW} applies one objective to every task. Evaluator-side anchor selection prefers a valid starter and falls back to a frozen reference. The agent never receives reference source or measurements. Let L , p , and II denote cycle latency, effective period, and initiation interval; a and c mark the anchor and candidate. The effective period is $p_x = \max(p_{\text{target}}, p_{\text{estimated}, x})$. The performance and area ratios are as follows. The weighted II branch applies when the task marks II relevant and both designs report positive II .

$$r_L = \frac{p_a L_a}{p_c L_c}, \quad r_P = \begin{cases} r_L^{0.85} (II_a / II_c)^{0.15}, & \text{reliable } II, \\ r_L, & \text{latency only,} \end{cases} \quad (3)$$

For resource counts A_j and C_j , define

$$\begin{aligned} \mathcal{R} &= \{\text{LUT, FF, DSP, BRAM}_{18\text{K}}, \text{URAM}\}, \\ \mathcal{R}_+ &= \{j \in \mathcal{R} : A_j > 1 \vee C_j > 1\}, \\ \hat{\mathcal{R}} &= \begin{cases} \mathcal{R}_+, & \mathcal{R}_+ \neq \emptyset, \\ \mathcal{R}, & \mathcal{R}_+ = \emptyset, \end{cases} \\ r_A &= \left[\max_{j \in \hat{\mathcal{R}}} \frac{\max(C_j, 1)}{\max(A_j, 1)} \right]^{-1}. \end{aligned} \quad (4)$$

TODO-FIGURE: Insert the VCL architecture diagram specified in the figure design brief.

Fig. 1. The VCL grants promotion authority to the validation chain and the hardware-quality rule. Failed or unaffordable candidates cannot replace the verified fallback.

The score maps a weighted hardware ratio through a bounded utility:

$$h = r_P^{0.55} r_A^{0.45}, \quad Q_{\text{HW}} = 1 - \frac{1}{(1 + h)^2}, \quad (5)$$

$$E = \max(0.80, 1 - 0.10u_{\text{credit}} - 0.10u_{\text{time}}),$$

$$S = 100VQ_{\text{HW}}E. \quad (6)$$

Here u_{credit} and u_{time} denote consumed fractions clipped to $[0, 1]$; E supplies the bounded budget term in the official score S . The promotion rule compares Q_{HW} , after $V(c) = 1$, and excludes E . It rejects a cycle reduction when clock degradation or the largest resource-growth ratio outweighs the gain.

IV. EVIDENCE REUSE FOCUSES EACH ITERATION

A. QoR-RAG Retrieves Compatible Actions

QoR-RAG builds a query from source metadata, baseline QoR, synthesis diagnostics, resource headroom, rejection history, and the task description. Its deterministic ranker weights token overlap by 3, 2, 4, 2, 3, 1 in that order. Keyword indicators map the query to reduction, memory-port, dataflow, stream, and factorization families. Each query returns at most one curated rule, one verified success, and one measured failure. Measured cases must match the target part and Vitis version. Workload guards compare the task description with family tags such as AES, GEMM, stencil, and dot product. Provenance checks reject paths that contain hidden, reference, or evaluator components. A 1,800-token cap bounds the retrieved context.

B. Failure Reflection Converts Errors into Constraints

Failure Reflection extracts the evidence that the next iteration needs. It removes ANSI control text, redacts paths and secrets, deduplicates lines, and selects bounded diagnostics that contain compiler, mismatch, timing, stream, deadlock, or timeout signals. The resulting record contains the failure class, candidate diff, implicated pragmas, loops, arrays, and the next constraint. A normalized signature counts repeated failure patterns. The next prompt and QoR-RAG query both consume this record, so Failure Reflection uses zero extra LLM requests.

A candidate fingerprint removes comments, normalizes whitespace, and removes optional braces around a single-statement `for` loop. The controller uses this fingerprint to reject unchanged or repeated candidates before tool execution.

TABLE I
CLAIM-ALIGNED RESULTS UNDER THE SAME TASKS AND BUDGETS.

TODO-TABLE: Insert three row groups: validation, QoR decisions, and evidence-reuse ablations.

It stops on budget exhaustion, duplicate candidates, or search stagnation.

V. EVALUATION MAPS EACH CLAIM TO EVIDENCE

A. VCL Preserves Required Validity

This test audits every proposed candidate, its gate vector, the corresponding promotion decision, and the final verified fallback. It lists hidden acceptance separately when the evaluator returns it.

Takeaway. Insert the measured rate at which the VCL blocks gate-invalid promotions.

B. Q_{HW} Selects Hardware Gains

This test compares the frozen anchor with accepted and rejected candidates. It reports latency, II , clock, all five resources, and Q_{HW} .

Takeaway. Insert one measured case where Q_{HW} accepts or rejects a nominal cycle gain.

C. Evidence Reuse Reduces Repetition

This test compares the full agent with variants that remove QoR-RAG, measured failure cases, or Failure Reflection. It reports success, repeated failures, LLM tokens, tool calls, credits, and wall time.

Takeaway. Insert the measured change in repeated failures and budget use.

All runs use Docker, Vitis 2025.2, and the Alveo U55C target.

VI. EVIDENCE CONTROLS PROMOTION

The VCL binds LLM proposals to the validation chain and Q_{HW} promotion rule. QoR-RAG supplies compatible evidence, and Failure Reflection supplies the next constraint without an extra LLM request. The final evaluation will measure how these mechanisms affect validity, hardware quality, and budget use.

REFERENCES

- [1] Advanced Micro Devices, Inc., *Vitis High-Level Synthesis User Guide*, AMD, 2025, version 2025.2.
- [2] C. Xiong, C. Liu, H. Li, and X. Li, “HLSPIlot: LLM-based high-level synthesis,” *arXiv preprint arXiv:2408.06810*, 2024.
- [3] H. Xu, H. Hu, and S. Huang, “Optimizing high-level synthesis designs with retrieval-augmented large language models,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.

APPENDIX A IMPLEMENTATION CONTRACT

The submission pipeline stores the candidate, gate results, best synthesis evidence, verified fallback, accounting, and terminal status in one run state. Every repair and optimization path uses the same validation chain and promotion rule. The output writer saves the latest kernel that passed every available required gate; failed and interrupted paths return the verified fallback.

APPENDIX B EXPERIMENTAL CONFIGURATION

APPENDIX C PER-TASK EVIDENCE

APPENDIX D TOKEN AND TOOL ACCOUNTING

APPENDIX E REPRODUCTION PROCEDURE

Evaluators build the supplied Docker image, configure the open-source model endpoint through environment variables, and mount the task and output directories. The command-line entry point writes the final kernel, structured run report, and submission evidence under the selected output root. The Vitis child environment excludes credentials, and report writers redact sensitive text.

TABLE II
REPRODUCTION CONFIGURATION FOR THE FINAL EXPERIMENT SET.

TODO-TABLE: Insert the final container, source revision, open-source model, inference settings, task budgets, and benchmark configuration.

TABLE III
PER-TASK CORRECTNESS AND HARDWARE EVIDENCE.

TODO-TABLE: Insert per-task gates, latency, II, clock, resources, Q_{HW} , status, and evidence identifiers.

TABLE IV
PER-STAGE MODEL AND TOOL ACCOUNTING.

TODO-TABLE: Insert per-stage prompt/completion tokens, LLM requests, tool calls, credits, and wall time.